

FUNDAMENTOS Y APLICACIONES DE BLOCKCHAINS

Homework 2

Depto. de Computación, UBA, 2do. Cuatrimestre 2025

9/10/25

Student:

Due: 21/10/25, 15:00 hs

Instructions

- Upload your solution to Campus; make sure it's only one file, and clearly write your name on the first page. Name the file '<your last name>_HW3.pdf.'

If you are proficient with $\text{\LaTeX}$, you may also typeset your submission and submit in PDF format. To do so, uncomment the "`%\begin{solution}`" and "`%\end{solution}`" lines and write your solution between those two command lines.

- Your solutions will be graded on *correctness* and *clarity*. You should only submit work that you believe to be correct.
- You may collaborate with others on this problem set. However, you must **write up your own solutions** and **list your collaborators and any external sources (including ChatGPT and similar generative AI chatbots)** for each problem. Be ready to explain your solutions orally to a member of the course staff if asked.

This homework contains 4 questions, for a total of 65 points.

1. Bitcoin transactions:

- (a) (7 points) Describe the mechanism used in the actual Bitcoin application that the miners follow in order to insert transactions into a block. Would the Consistency (aka Persistence) and Liveness properties we saw in class, as well as the V, I, R functions have to be modified to capture the actual mechanism? Elaborate.

Solution:

Función V : el objetivo es validar un bloque. En el paper no se especifican algunos detalles que puede ser útil mostrar para especificar qué es un bloque válido en la red de Bitcoin (tomo la función V de la clase, la de "Public transaction ledger protocol"). Para que un bloque sea válido en la red de bitcoin, debe cumplir las siguientes propiedades.

- La suma total de lo que se envía en la transacción coinbase debe ser la correcta dada la recompensa actual de la red. Además, la transacción coinbase debe ser la primera del bloque.
- Las demás transacciones del bloque deben ser válidas. Es decir, para cada transacción:
 - La firma digital es correcta respecto a quien envía la transacción.
 - No hay double spends: la suma total de lo que se envía en esa transacción es menor o igual a los fondos de las cuentas a debitar.
 - Si la cuenta a debitar viene de un UTXO de una coinbase transaction, entonces ese UTXO debe haber sido creado con 100 bloques de antelación. (referencia)
- El PoW es correcto: se cumple que el hash del bloque con ese nonce es menor al target.
- El hash del bloque anterior es correcto.

Función R : El objetivo es leer una cadena de bloques. No hacen falta modificaciones. Al leer una cadena de bloques, solamente se verifica que sea válida.

Función I : El objetivo es, dado un input tape, seleccionar las transacciones que se van a incluir en el nuevo. El criterio de selección es que sean válidas respecto al ledger (x_C) (y que no estén en este ledger). Las modificaciones que tendríamos que hacer para reflejar el protocolo de bitcoin son:

- El "input tape" se llama *mempool*. Ahí se guardan las transacciones que los mineros escuchan de la red. Cada minero tiene la suya, y entre las *mempools* de los mineros puede haber ligeras diferencias.

- El criterio de selección (además de los que se detallan arriba, que son necesarios para mantener la validez del ledger) se basa en incentivos económicos: los mineros van a elegir las transacciones que mejores comisiones tengan (*transaction fees*).
- La "transacción neutra" que se agrega al principio de la subcadena mencionada en la definición del paper se reemplaza por la transacción coinbase, la que usan los mineros para recibir la recompensa por haber minado el bloque.

Propiedad de *Consistency*: Esta propiedad muestra que, una vez que un minero (honesto) reporta una transacción tx_0 lo suficientemente lejos del final de la cadena (es decir, ya hubo una cantidad k de bloques posteriores al bloque que tiene esta transacción tx_0), esa transacción va a ser reportada como estable por todos los mineros, en exactamente la misma posición. Esta es la razón por la cual las transacciones se consideran irreversibles en la red de bitcoin. Esta propiedad no necesita de modificaciones para reflejar el protocolo de bitcoin. Podemos agregar que en bitcoin se llaman "confirmaciones" a la cantidad de bloques que tiene esa transacción por delante. En particular, una vez que $k = 6$, la transacción se considera irreversible.

Propiedad de *Liveness*: En el paper, esta propiedad muestra que, dada una transacción válida, si los mineros la reciben continuamente (durante u rondas), eventualmente la van a reportar como estable (a más de k bloques del final de la cadena). Para adaptarla al protocolo de bitcoin, es importante agregarle que la transacción, además de ser válida, tiene que tener un precio competitivo de *transaction fees*. Esto se debe a que los mineros siempre van a elegir las transacciones con mejor comisión. Si la transacción está muy por debajo del promedio de transacción aceptada por los mineros, no la van a incluir en el bloque, dado que un bloque tiene un tamaño limitado. Por ende, no se cumpliría liveness si no se agrega esta restricción.

(b) (3 points) Describe the purpose of a *coinbase* transaction.

Solution: Las transacciones de tipo coinbase sirven para recompensar a los mineros por haber minado el bloque con éxito; quien mina el bloque tiene el derecho de agregar esta transacción a la blockchain, en la que recibe esta recompensa.

Esta transacción es distinta porque no tiene origen, y no son los mismos fondos que las comisiones que puedan cobrar los mineros. Esta es una recompensa que recibe por haber minado el bloque. Además, cumple el propósito de crear nuevas monedas para que circulen en la red.

Nota: para escribir la solución del ejercicio 1, tomé en cuenta las diapositivas de la clase, el paper de "Bitcoin backbone" y bitcoin.org.

2. Proofs of work:

- (a) (5 points) Describe the purpose and implementation of the **2x1 PoW** technique.

Solution: El propósito de usar la técnica 2x1 PoW es lograr que en el *1/2 Consensus Protocol*, que requiere que se ejecuten dos PoWs diferentes, se puedan ejecutar de a una sola vez. Esto es para prevenir que un adversario pueda ejecutar solo uno de los PoW en paralelo, pudiendo pasar su *hashing power* de uno al otro.

Esto se debe a que si no se hiciera así, se tendrían que ejecutar dos loops distintos, sumando un total de $2q$ consultas. De este modo, se combinan ambas tareas en una sola, resultando en q consultas.

La implementación se basa en ejecutar solo un hash en vez de dos. En el *1/2 Consensus Protocol* se usaban dos funciones hash, H_0 y H_1 . En la función `double_pow` se usa solo una función de hash, haciendo que se tome primero un valor como el output de la función de hash "tradicional" y el otro va a ser el reverso de ese hash.

Se agrega una etiqueta a las tuplas existentes, para diferenciar cuál PoW es cuál.

Notar que en la función de `double_pow` se puede lograr un PoW para uno de los dos, para los dos, o para ninguno.

- (b) (7 points) Let T_1 and T_2 be the target values in 2x1 PoW, and κ the size of the hash function's output. What relation should they satisfy for the technique to work? Elaborate.

Solution: Sabemos que T_1, T_2 van a ser de κ bits. Vamos a pedir que $T_1 = 2^{t_1}$ y $T_2 = 2^{t_2}$, donde t_1 y t_2 son enteros positivos. Para cumplir la primer prueba de `double_pow`, tenemos que pedir los $\kappa - t_1$ bits más significativos del output de la función de hash deben ser 0. Esto es porque el output de la función de hash debe ser menor a T_1 . Análogo para la segunda prueba, solamente que tenemos que pedir que los bits **menos** significativos sean cero.

Entonces, dado un hash u , necesitamos que sus primeros $\kappa - t_1$ bits sean cero, y sus últimos $\kappa - t_2$ bits sean cero.

Entonces, para que estas dos condiciones sean independientes, vamos a necesitar que $\kappa - t_1 + \kappa - t_2 \leq \kappa$. Esto es porque queremos que los bits que le afectan al primer PoW no afecten al segundo, Simplificando queda $t_1 + t_2 \geq \kappa$. Entonces, nos quedan dos "particiones" de los bits del hash.

Vemos que, si T_1 y T_2 no fuesen potencias de dos, terminaríamos imponiendo una condición sobre la otra partición de los bits. Por ejemplo, si $T_1 = 00101000$ (40), $T_2 = 00001000$ (8) y $u = 000xxxxx$ sabemos que es válido, pues es menor

que T_1 . Ahora, $u = 00100xxx$ también es válido, pero el problema es que ya estamos imponiendo una condición sobre la otra partición de los bits, los que van a corresponder con la prueba del reverso. Esta prueba va a requerir que los bits menos significativos sean cero, entonces la estamos condicionando. Entonces, si T_1 y T_2 no fuesen potencias de dos, terminaríamos imponiendo una condición sobre la otra partición de los bits. Por ende, para mantener la independencia estadística entre las pruebas, vamos a necesitar que T_1 y T_2 sean potencias de dos.

Otra cosa a notar es que hay un "balance" entre t_1 y t_2 . Si uno es muy pequeño, el otro tiene que ser muy grande, ya que la suma de ambos tiene que ser mayor a κ . Esto nos condiciona a que si queremos que un PoW sea más fácil, el otro tiene que compensar esa diferencia.

- (c) (8 points) Design and argue correctness of an $\ell \times 1$ PoW scheme. Note that such a scheme would enable an ℓ -parallel blockchain. What properties of the blockchain or ledger application would, if any, benefit from a parallel blockchain? Elaborate.

Solution:

Para diseñar un esquema $\ell \times 1$ PoW, vamos a necesitar particionar el hash en ℓ partes, de manera que todas sean estadísticamente independientes.

Digamos que son todas partes iguales para simplificarlo. Decimos entonces que tienen tamaño t , haciendo que $\kappa = \ell \cdot t$.

Luego, queremos que las ℓ particiones del hash sean menores a un target $T \leq 2^t$. Necesitamos pedir esto, ya que sería trivial pedir que un número binario de exactamente t bits sea menor a 2^t .

Como cada prueba mira su propia partición de bits, y ninguna partición se superpone con otra, las pruebas son estadísticamente independientes.

Una de las propiedades que podría verse afectada es chain growth. Como los mineros trabajan en ℓ blockchains a la vez, cada una de ellas se vería afectada. La red en su conjunto seguiría produciendo bloques a más o menos la misma velocidad. Pero, ahora esos bloques se van a distribuir en cada una de las ℓ cadenas.

Vemos que liveness se ve afectada. Depende de *Chain Growth* y *Chain Quality*. La cantidad de rondas u que se necesitan para que todos los mineros honestos incluyan esa transacción en el bloque (con profundidad k) va a tener que ser ℓ veces mayor, puesto que se distribuye el *hashing power* en ℓ cadenas.

3. **Strong consensus:** We saw how the Bitcoin backbone protocol can be used to solve the *consensus* problem (aka *Byzantine agreement*). In the *strong consensus* problem, the *Validity* condition is strengthened to require that the output value be one of the honest parties' inputs—this property is called *Strong Validity*. (Note that this distinction is relevant only in the case of non-binary inputs.)

- (a) (5 points) What should be the assumption on the adversarial computational power (similar to the “Honest Majority Assumption”) for Strong Validity to hold?

Solution: Si tomamos m como la cantidad de inputs posibles, tenemos que restringir el *hashing power* de los adversarios por un factor de $m - 1$. Entonces, el *assumption* que tenemos que hacer es $t \leq \frac{1-\delta}{m-1} \cdot (n - t)$. Usamos $m - 1$ ya que en el caso de $m = 2$ queríamos dejarlo igual que antes.

Esta cota es suficiente. Pongamos el caso de que los jugadores honestos estan divididos en los primeros $m - 1$ valores, y el adversario elige el m -ésimo valor. Querríamos que el adversario no pueda ganarle en votos a incluso la división con menos jugadores de los honestos. Digamos que los honestos se dividen equitativamente (el peor caso, ya que así no tienen una mayoría abrumadora de votos), luego habría $\frac{n-t}{m-1}$ votos para esa división. El adversario tendría que tener al menos $\frac{n-t}{m-1}$ votos para ganarle, lo cual es imposible si $t \leq \frac{1-\delta}{m-1} \cdot (n - t)$.

- (b) (5 points) State and prove the Strong Validity lemma.

Solution: Dado un protocolo BA que opera sobre un conjunto de valores de entrada V , con $|V| = m$, y bajo la suposición de que $t \leq \frac{1-\delta}{m-1} \cdot (n - t)$, asumiendo $k = O(\kappa)$ y $L = \Omega(\kappa)$, se satisface Strong Validity con probabilidad $1 - e^{-\Omega(\kappa)}$.

Strong Validity nos dice que tenemos que el output (valor elegido) sea uno de los inputs honestos. Para asegurarnos esto usamos la suposición anterior, que nos asegura que mantenemos el hashing power del protocolo anterior.

Como vemos en la demostración anterior de Validity, vale que $C^{\lceil k}$ tiene al menos k bloques, y luego vale Chain quality con la misma fracción de bloques adversarios.

4. **Smart contract programming:** In this assignment you will create your **own custom token**. Your contract should implement the public API described below:

- **owner:** a public payable address that defines the contract's "owner," that is, the user that deploys the contract
- *Transfer(address indexed from, address indexed to, uint256 value):* an event that contains two addresses and a uint256
- *Mint(address indexed to, uint256 value):* an event that contains an address and a uint256
- *Sell(address indexed from, uint256 value):* an event that contains an address and a uint256
- **totalSupply():** a view function that returns a uint256 of the total amount of minted tokens
- **balanceOf(address account):** a view function returns a uint256 of the amount of tokens an address owns
- **getName():** a view function that returns a string with the token's name
- **getSymbol():** a view function that returns a string with the token's symbol
- **getPrice():** a view function that returns a uint128 with the token's price (at which users can redeem their tokens)
- **transfer(address to, uint256 value):** a function that transfers *value* amount of tokens between the caller's address and the address to; if the transfer completes successfully, the function emits an event *Transfer* with the sender's and receiver's addresses and the amount of transferred tokens and returns a boolean value (*true*)
- **mint(address to, uint256 value):** a function that enables *only the owner* to create value new tokens and give them to address to; if the operation completes successfully, the function emits an event *Mint* with the receiver's address and the amount of minted tokens and returns a boolean value (*true*)
- **sell(uint256 value):** a function that enables a user to sell tokens for wei at a price of *600 wei per token*; if the operation completes successfully, the sold tokens are removed from the circulating supply, and the function emits an event *Sell* with the seller's address and the amount of sold tokens and returns a boolean value (*true*)
- **close():** a function that enables *only the owner* to destroy the contract; the contract's balance in wei, at the moment of destruction, should be transferred to the owner's address
- **fallback** functions that enable anyone to send Ether to the contract's account
- **constructor** function that initializes the contract as needed

You should implement the smart contract and deploy it on the course's Sepolia Testnet. Your contract should be as secure and gas efficient as possible. After deploying your contract, you should buy, transfer, and sell a token in the contract.

Your contract should implement the above API **exactly as specified**. *Do not* omit implementing one of the above variables/functions/events, do not change their name or parameters, and do not add other public variables/functions. You can define other private/internal functions/variables, if necessary.

You should provide:

- (a) (5 points) A detailed description of your high-level design decisions, including (but not limited to):
- What internal variables did you use?
 - What is the process of buying/selling tokens and changing the price?
 - How can users access their token balance?

Solution: Decidí extender levemente la funcionalidad del contrato para que tenga más usabilidad. En particular, agregué la función buy, que permite a los usuarios comprar tokens con el ether que ya habían depositado en el contrato. Estos tokens se los compran directamente a la cuenta del contrato, y luego se transfieren al usuario. Esto evita que el usuario tenga que enviar ether al contrato y luego hablar con el propietario para que le mintee los tokens.

Además, decidí agregar una función changePrice que permite cambiar el precio de los tokens y una función etherBalanceOf que permite a los usuarios ver su balance de ether (además de balanceOf para su balance de tokens).

Variables internas: owner, tokenBalance, _totalSupply, price y finalmente etherBalance. Si el precio no necesitaba ser modificado, podría haberlo dejado como una constante, pudiendo obtenerlo mediante la función getPrice. Sin embargo, como la especificación mencionaba "cómo cambiar el precio", agregué la función changePrice, que solo puede ser llamada por el propietario del contrato. El mapping tokenBalance guarda para cada dirección el saldo de la cuenta. El mapping etherBalance guarda para cada dirección el monto de ether recibido por la misma. Esto permite a los usuarios enviar ether al contrato, y luego usarlo para comprar tokens. Al ser en dos pasos, los usuarios podrían tener un balance de ether y hacer varias transacciones para comprar tokens (por ejemplo, si quisieran especular con el precio, y hacer compras y ventas en diferentes momentos para ganar con la diferencia de precio). Luego, para retirar su ether, usan la función withdraw.

Para comprar tokens, los usuarios usan la función buy. Esto permite a los usuarios comprar tokens con el ether que ya habían enviado al contrato, que se guarda en etherBalance. Luego, el usuario le compra los tokens al

contrato directamente. Esto es una decisión de diseño. Consideré que era más simple que le compren al contrato (asumiendo que el contrato tiene tokens en su balance, generados por el propietario previamente).

Para vender tokens, los usuarios llaman a `sell`. Esto genera un evento `Sell` con la dirección del vendedor y la cantidad de tokens vendidos. Luego, los usuarios pueden retirar su ether mediante la función `withdraw`. El hecho de que hayamos definido que los usuarios pueden guardar su ether en nuestro contrato permite que el `withdraw` no se haga en el momento de la llamada a `sell`, sino que se pueda hacer después. Esto nos da más seguridad en caso de que se revierta la ejecución.

Los usuarios acceden a su balance de tokens mediante la función `balanceOf`. Además, acceden a su balance de ether mediante la función `etherBalanceOf`.

(b) (5 points) A detailed gas evaluation of your implementation, including:

- The cost of deploying and interacting with your contract.
- Techniques to make your contract more cost effective.

Solution: Gas de las interacciones:

- Deploy: 1,294,156 gas
- sending ether via `receive()`: 43,450 gas
- buy: 56,771 gas
- sell: 54,353 gas
- withdraw: 29,473 gas

Para hacer que el contrato gaste menos gas, se podrían hacer ciertos recortes. Por ejemplo, en vez de permitir que se guarde el balance de ether, en otro arreglo, podríamos hacer que los usuarios envíen el ether directamente en la llamada a `buy`, y que al vender se les transfiera el ether resultante directamente a su dirección. El costo de gas sería menor ya que tendría menos escrituras y lecturas de memoria (y menos llamadas en general), aunque el contrato se vuelve menos seguro y pierde flexibilidad.

(c) (5 points) A thorough listing of potential hazards and vulnerabilities that can occur in the smart contract and a detailed analysis of the security mechanisms that can mitigate these hazards.

Solution: Los principales problemas de seguridad están relacionados con los privilegios que tiene el propietario del contrato. Para empezar, el dueño del contrato puede retirar todo el ether del contrato, lo que haría que los usuarios no puedan vender sus tokens. Además, el dueño puede manipular arbitrariamente el precio de los tokens.

Por estas razones el contrato no es del todo justo para los usuarios, que corren el riesgo de perder su ether si el contrato se cierra.

Para mitigar estos problemas, se podría hacer que el dueño del contrato no pueda retirar el ether de otros usuarios, solo de su propia cuenta. También que no pueda cambiar el precio de los tokens arbitrariamente. Hay varias soluciones a esto, una es que el dueño del contrato pueda cambiar el precio, pero que haya un delay entre que lo propone y que se hace efectivo. Si no, podría pasar que el dueño del contrato vea que alguien esta por vender tokens, y le baje el precio para que, al vender, el usuario reciba menos ether. También se podría hacer que los usuarios que quieren comprar/vender sus tokens pongan un precio mínimo/máximo que aceptan (en esa transacción).

- (d) (5 points) The transaction history of the deployment of and interaction with your contract.

Solution:

- Deploy.
- sending ether via receive().
- buy.
- sell.
- withdraw.

- (e) (5 points) The code of your contract.

Solution:

```
// SPDX-License-Identifier: GPL-3.0

// This makes sure we use a solidity version
// that automatically handles overflow/underflow
// for me :D. It reverts the operation if it
// overflows/underflows.
pragma solidity ^0.8.0;
```

```

contract Token12 {
    address public owner;
    mapping(address => uint256) public tokenBalance;
    mapping(address => uint256) public etherBalance;
    uint256 public _totalSupply;
    uint128 public price;

    event Transfer(
        address indexed from,
        address indexed to,
        uint256 value
    );
    event Mint(address indexed to, uint256 value);
    event Sell(address indexed from, uint256 value);
    event Buy(address indexed from, uint256 value);

    // podria agregar eventos Withdraw, Buy, PriceChanged

    constructor() {
        owner = msg.sender;
        _totalSupply = 0;
        price = 600 wei;
    }
    function getName()
        public
        pure
        returns (string memory)
    {
        return "Token12";
    }

    function getSymbol()
        public
        pure
        returns (string memory)
    {
        return "T12";
    }

    function getPrice() public view returns (uint128) {
        return price;
    }
}

```

```

    }

    function changePrice(uint128 newPrice) public {
        require(msg.sender == owner);
        require(newPrice > 0);
        price = newPrice;
    }

    function totalSupply() public view returns (uint256) {
        return _totalSupply;
    }

    function balanceOf(address account)
        public
        view
        returns (uint256)
    {
        return tokenBalance[account];
    }

    function etherBalanceOf(address account)
        public
        view
        returns (uint256)
    {
        return etherBalance[account];
    }

    // This is to avoid code duplication on
    // 'transfer' and 'buy'
    function _transfer(
        address from,
        address to,
        uint256 value
    )
        internal
    {
        require(tokenBalance[from] >= value);
        require(value > 0);
        require(to != address(0));
        require(to != from);
    }

```

```

        tokenBalance[from] -= value;
        tokenBalance[to] += value;

        emit Transfer(from, to, value);
    }

    function transfer(
        address to,
        uint256 value
    )
        public
        returns (bool)
    {
        _transfer(msg.sender, to, value);
        return true;
    }

    function mint(
        address to,
        uint256 value
    )
        public
        returns (bool)
    {
        require(msg.sender == owner);
        require(to != address(0));
        require(value > 0);

        emit Mint(to, value);

        // update my state
        tokenBalance[to] += value;
        _totalSupply += value;

        return true;
    }

    function buy(uint256 value) public returns (bool) {
        // This is not creating new tokens, it's
        // just transferring tokens from the
        // contract itself to the sender.

```

```

        // However, the contract must have enough
        // tokens to transfer to the sender, which
        // means the contract owner must have
        // minted enough tokens for the contract
        // beforehand.

        require(value > 0);
        // sender must have paid enough ether to buy the tokens
        uint256 totalEther = value * price;
        require(totalEther <= etherBalance[msg.sender]);
        // the contract itself must have enough
        // tokens to sell to the sender
        require(tokenBalance[address(this)] >= value);

        etherBalance[msg.sender] -= totalEther;

        // this buys from the contract itself, so we use _transfer()
        _transfer(address(this), msg.sender, value);

        emit Buy(msg.sender, value);

        return true;
    }

    function sell(uint256 value) public returns (bool) {
        require(value > 0);

        // In order to sell, I need to have
        // enough tokens in the contract.
        uint256 totalEther = value * price;
        require(
            address(this).balance >= totalEther,
            "Ether unavailable. Contact owner."
        );

        // This is to avoid users calling sell()
        // when I (owner) have closed the
        // contract. However, they can still call
        // transfer() to transfer tokens to other
        // addresses, though they won't be able
        // to sell them.

```

```

        require(
            _totalSupply >= value,
            "Not enough tokens. Contact owner."
        );
        require(tokenBalance[msg.sender] >= value);

        // overflow is handled by ^0.8.0
        etherBalance[msg.sender] += totalEther;

        emit Sell(msg.sender, value);

        // update my state
        tokenBalance[msg.sender] -= value;
        _totalSupply -= value;

        return true;
    }

    function withdraw(uint256 value)
        public
        returns (bool)
    {
        require(value > 0);
        require(etherBalance[msg.sender] >= value);

        // update state before external call
        etherBalance[msg.sender] -= value;

        payable(msg.sender).transfer(value);

        return true;
    }

    function close() public {
        require(msg.sender == owner);
        payable(owner).transfer(address(this).balance);
        _totalSupply = 0;

        // I don't reset tokenBalance here,
        // just to keep track of the holders
        // (in case I'd fund this contract again

```

```
        // so holders can still use their tokens)
    }

    fallback() external payable {
        // save the amount of ether received
        // by the sender
        etherBalance[msg.sender] += msg.value;
    }

    receive() external payable {
        // save the amount of ether received
        // by the sender
        etherBalance[msg.sender] += msg.value;
    }
}
```

